COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S4
DATE PERFORMED:	October 07, 2026	**DATE SUBMITTED:**	October 00, 2026
NAME:	Quiming, Kadaphy James A. Larracas, Jerry Matthew L.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 11 BASIC ALGORITHM ANALYSIS

1. PROCEDURE OUTPUT

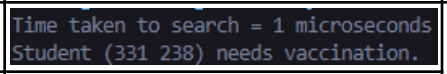
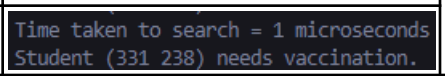
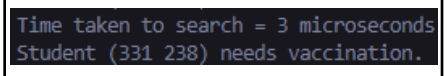
ILO A: Measure the runtime of algorithms using theoretical analysis

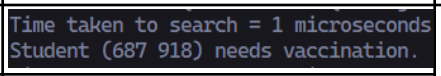
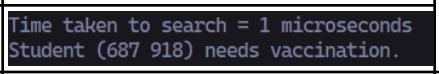
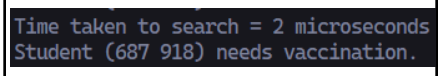
ANALYSIS

The number of operations keeps increasing (from 310 to 510) with a constant difference of 200 (510 - 310 = 200) this suggests that it is a quadratic function. Having known what kind of function the graph is, we can tell that the worst case is $O(n^2)$, it happens when every element must be compared. While the best case is O(1) when the size of the array is found immediately..

ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

LARRACAS

Input Size (N)	Execution Speed (µsec)	Screenshot	Observation(s)
1000	1 µsec		Both input sizes took a time taken to search for only 1 microsecond, the same with my partner device.
10000	1 µsec		
100000	3 µsec		The time taken to search in my pc was 3 microseconds and only has a 1 difference to Quiming's time taken to search and much faster than my device since he got 2 microseconds only to search.

QUIMING			
Input Size (N)	**Execution Speed (µsec)**	**Screenshot**	**Observation(s)**
1000	1 µsec		Both only took 1 µsec showing fast and consistent execution and having the same performance as my partner's device.
10000	1 µsec		
100000	2 µsec		The time it took to execute increases to 2 µsec, showing a align increase in execution time, but it shows that my device is slightly faster than my partner's device.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

LARRACAS

ILO A: Measure the runtime of algorithms using theoretical analysis

PROBLEM 1: Consider a program that returns true if the elements in an array are unique.

THEORETICAL ANALYSIS

The algorithm has a worst case complexity of O(n^2). The nested loops compares every pair of elements in which it takes n(n - 2) / 2 comparisons when all elements are duplicated in the last pair. And the best case is O(1) because the first two elements are equal.

EXPERIMENTAL ANALYSIS

The execution time increased as the input size increased.

ANALYSIS AND COMPARISON

The experimental results consist the theoretical which is the O(n^2) complexity, therefore doubling n multiplied the runtime by about 4 in which it is expected behavior of a quadratic algorithm. In this case, both analysis shows algorithm becomes significantly slower as the input size grows.

PROBLEM 2: Consider another program that raises a number x to an arbitrary nonnegative integer, n.

THEORETICAL ANALYSIS
The rpower reduces n by 1 on each call, so T(n-1), which gives O(n) multiplications. On the other hand the brpower halves the n on each call, so T(n / 2), which gives O n log n multiplications.
EXPERIMENTAL ANALYSIS
The rpower's time grew roughly in proportion to n and brpower's time had a changed a little bit, rising only slightly even as n grew.
ANALYSIS AND COMPARISON
The experiment matches what the theory predicted. When n gets bigger, rpower takes more time in a steady, straight line way in which it fits O(n). On the other hand, brpower slows down a little bit as n grows, which it has O(n log n). Because the bigger n gets, the larger the difference between the two becomes.

ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

PROBLEM 1: Consider a program that returns true if the elements in an array are unique.
THEORETICAL ANALYSIS
The algorithm has a worst-case complexity of **$O(n^2)$** because it uses nested loops to compare every possible pair of elements.
EXPERIMENTAL ANALYSIS
The execution time increased as the input size increased. The increase in time became more noticeable for larger input sizes because the number of comparisons grows very quickly.
ANALYSIS AND COMPARISON
The experimental results are consistent with the theoretical **$O(n^2)$** complexity. Both show that the algorithm becomes significantly slower as the input size grows.

PROBLEM 2: Consider another program that raises a number x to an arbitrary nonnegative integer, n. (**rpower**)
THEORETICAL ANALYSIS
rpower has a time complexity of **O(n)** because it makes one recursive call for each value of n.
EXPERIMENTAL ANALYSIS
The runtime increased as n increased because more recursive calls were required.
ANALYSIS AND COMPARISON
The experimental results support the theoretical O(n) complexity. The runtime grows approximately in proportion to the input value, so as the input value grows, the runtime also grows with it.

QUIMING

ILO A: Measure the runtime of algorithms using theoretical analysis

PROBLEM 1: Consider a program that returns true if the elements in an array are unique.

THEORETICAL ANALYSIS

The Unique(A) algorithm uses two nested for loops. The first loop selects an element, while the second loop compares it with all succeeding elements.

- Best Case: O(1) – When a duplicate is found immediately
- Worst Case: $O(n^2)$ – When all elements are unique and every pair is compared.

EXPERIMENTAL ANALYSIS

The program was tested using different input sizes. The higher the input size is, the longer it is to execute. Bigger arrays need more comparisons, especially when there are no duplicate elements found in the array.

ANALYSIS AND COMPARISON

The experimental results generally agree with the theoretical analysis. The runtime increases as the input size grows, which is expected for $O(n^2)$ runtime. The worst case requires the most comparisons because the algorithm must check every pair possible.

PROBLEM 2: Consider another program that raises a number x to an arbitrary nonnegative integer, n.

THEORETICAL ANALYSIS

The brpower reduces the n by half every recursion which mean its complexity is **O(log n)**

EXPERIMENTAL ANALYSIS

The larger the value of n, the longer the execution time is. Comparing rpower with brpower, brpower required significantly fewer recursive calls.

ANALYSIS AND COMPARISON

The experimental results support the theoretical O(log n) complexity. Because n is repeatedly divided by two, brpower is much faster and more efficient than rpower, especially if the value of n gets larger.

ILO B: Analyze the results of runtime performance by comparing experimental and theoretical analysis

PROBLEM 1: Consider a program that returns true if the elements in an array are unique.

THEORETICAL ANALYSIS

The algorithm has a worst-case complexity of **$O(n^2)$** because it uses nested loops to compare every possible pair of elements.

EXPERIMENTAL ANALYSIS

The execution time increased as the input size increased. The increase in time became more noticeable for larger input sizes because the number of comparisons grows very quickly.

ANALYSIS AND COMPARISON
The experimental results are consistent with the theoretical **O(n²)** complexity. Both show that the algorithm becomes significantly slower as the input size grows.
PROBLEM 2: Consider another program that raises a number x to an arbitrary nonnegative integer, n. (**rpower**)
THEORETICAL ANALYSIS
rpower has a time complexity of **O(n)** because it makes one recursive call for each value of n.
EXPERIMENTAL ANALYSIS
The runtime increased as n increased because more recursive calls were required.
ANALYSIS AND COMPARISON
The experimental results support the theoretical O(n) complexity. The runtime grows approximately in proportion to the input value, so as the input value grows, the runtime also grows with it.
PROBLEM 2: Consider another program that raises a number x to an arbitrary nonnegative integer, n. (**brpower**)
THEORETICAL ANALYSIS
brpower has a time complexity of **O(log n)** because it reduces n by half in every recursive call.
EXPERIMENTAL ANALYSIS
The runtime increased as the value of n increased because more and more recursive calls would be required.
ANALYSIS AND COMPARISON
The experimental results agree with the theoretical O(log n) complexity. brpower is more efficient than rpower because it requires far fewer recursive calls.

CONCLUSION
LARRACAS
I think I did well in this activity because I learned how to compare theoretical complexity with actual run time and I also confirmed that the unique algorithm is O(n^2), that rpower is O(n) and that brpower is O(n log n) and my measured results matched these predictions. I also saw how much faster and efficient algorithm like brpower is when the input gets large. At first, I had trouble with the experimental analysis, especially in measuring runtimes and explaining why the numbers grew the way they did. And that's where I notice that I still need to improve myself by praciticing analyzing more algorithms and testing them with different input sizes so I can interpret the results more precisely and accurately.
QUIMING
● **Summary of lessons learned** – I learned that theoretical analysis predicts an algorithm's efficiency, while

experimental analysis measures its actual runtime.

- **Analysis of the procedure** – The diff algorithm has the complexity of $O(n^2)$ because it has to compare for each element of y. Binary search is O(log n) because it halves the searching range each call. The experimental results matched the theoretical results.
- **Analysis of the supplementary activity** – Unique algorithm's worst case is $O(n^2)$. rpower is O(n) while brpower is O(log n).
- **Concluding statement / Feedback: How well did you think you did in this activity? What are your areas for improvement?** – I learned how to actually compare theoretical complexity with actual runtime. I had trouble with the experimental and theoretical analysis.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **n/a**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **n/a**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **n/a**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.